

FRED TALKS

17th May 2026

CONTENTS

10 Lessons for Agentic Coding

DREW BREUNIG · 638 WORDS

Where the goblins came from | OpenAI

OPENAI · 1128 WORDS

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 1062 WORDS

The lies I used to tell myself

CATE HALL · 1900 WORDS

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 1053 WORDS

10 Lessons for Agentic Coding

DREW BREUNIG · 08 MAY 2026 · [SOURCE](#)

What should we do when code is cheap?



Lately, this blog has featured a lot of writing about agentic coding. Frontier models are *really* good at coding these days, much better than they are at other tasks. Coding with agents feels like a preview of the future, a playground for seeing how far we can push agent capabilities. It's invigorating, rewarding, and deeply *weird*.

I've been keeping a running list of tips for agentic coding: guidelines or rules one might give to someone just getting started with Codex, Claude Code, Pi, or any other agent. Ideally each tip is generalizable guidance, relevant to any agentic programming. I'm also looking for durable lessons that will stick around as models and harnesses improve.

Below is my current list: *10 Lessons for Agentic Coding*. Ten's a nice round number; a good time to put this out there.

To be clear: I take credit only for honing and compiling these guidelines. As [Kshetrajna Raghavan](#) said to me today, "It's *crazy* how we're all converging on similar lessons."

(If you think I've missed anything below, please reach out!)

10 Lessons for Agentic Coding

1. **Implement to learn.** You can go far with Spec-Driven Development, but the act of writing code surfaces decisions you hadn't considered and makes your spec better. When code is cheap, implement to learn.

2. **Rebuild often.** Implement early and *often* to learn more. Fork and recode crazy thought experiments. Find out how far you can take feature. Of course, you want to iterate and compound your efforts, but cheap code means you can reconnoiter and reinvent in ways you never could.
3. **Invest in end-to-end tests.** When we can reinvent our code cheaply, we should spend time writing tests that measure our product's *functions*, not *how* it performs them. We want behavioral contracts that grant us the freedom to rebuild and reimplement.
4. **Document intent.** Tests detail our goals while code encodes our methods, but neither captures the *why*. Your intent motivates your decisions, and persisting it alongside the code helps you and your agent compound those decisions in a consistent direction.
5. **Keep your specs in sync.** Update your specs, the markdown files containing your goals and plans, as you advance your code and your tests. Treating your spec as a frozen artifact written before work begins, you'll fail to capture learnings during implementation. Keeping it current lets it constantly inform your and your agents' choices, and makes frequent rebuilds easier.
6. **Find the hard stuff.** Work on a project long enough and things will stop being easy. You'll speed through the boilerplate work, the obvious design decisions, and start hitting the ugly, difficult work: intuitive design, performance, security, resilience, and systemic architecture. Anyone can vibe the easy stuff. The hard work is where the value is. Find it and dig in.
7. **Automate everything that's easy.** To spend more time on the hard stuff, minimize the time you spend on easy things. Distill learnings into skills, build loops, automate code reviews, and let your tools compound. But careful: don't get stuck in a Mystery House.
8. **Develop your taste.** When code arrives fast but feedback doesn't, the only source of feedback that keeps up is your own. The better you know your domain, your users, and their problems, the further you can go without checking in.
9. **Agents amplify experience.** Talented developers underestimate how much intuition they bring to their prompts: the right terms, the right framing, and the right level of specificity. If you know your stack, you can save countless cycles during both implementation and debugging, and cut down needless agent exploration. Pair technical expertise coupled with great taste for an unbeatable advantage.
10. **Code is cheap, but maintenance, support, and security aren't.** Agentic code is "free as in puppies." Support isn't cheap and neither is security. Build fast, but mind the maintenance you're adopting.

Where the goblins came from | OpenAI

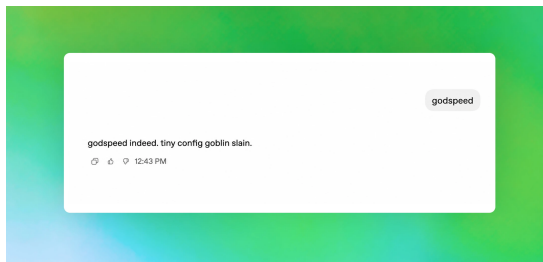
OPENAI · 02 MAY 2026 · [SOURCE](#)

April 29, 2026

[Publication](#)

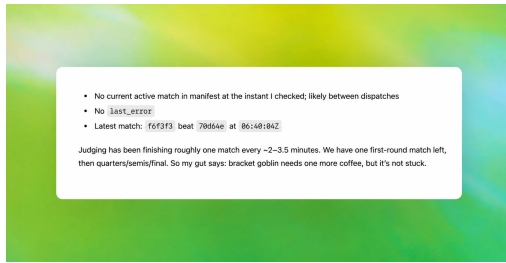
Share

Starting with GPT-5.1, our models began developing a strange habit: they increasingly mentioned goblins, gremlins, and other creatures in their metaphors. Unlike model bugs that show up through a tanking eval or a spiking training metric and point back to a specific change, this one crept in subtly. A single “little goblin” in an answer could be harmless, even charming. Across model generations, though, the habit became hard to miss: the goblins kept multiplying, and we needed to figure out where they came from.

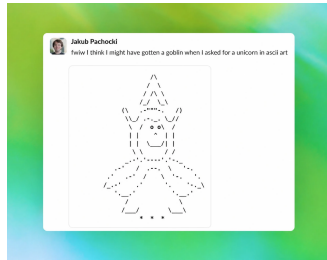


In early testing, GPT-5.5 in Codex showed an odd affinity for goblin metaphors.

The short answer is that model behavior is shaped by many small incentives. In this case, one of those incentives came from training the model for the [personality customization feature](#), in particular the Nerdy personality. We unknowingly gave particularly high rewards for metaphors with creatures. From there, the goblins spread.



The goblins were funny at first, but the increasing number of employee reports became concerning.



An interesting interaction our Chief Scientist had with GPT-5.5.

The first signs of creatures

The first time we clearly saw the pattern was in November, after the GPT-5.1 launch, although it may have started earlier. Users complained about the model being oddly overfamiliar in conversation, which prompted an investigation into specific verbal tics. A safety researcher had experienced a few “goblins” and “gremlins” and asked that they be included in the check. When we looked, use of “goblin” in ChatGPT had risen by 175% after the launch of GPT-5.1, while “gremlin” had risen by 52%.

A measurable small lexical quirk in GPT-5.1.

At the time, the prevalence of goblins did not look especially alarming. A few months later, the goblins came back to haunt us in a much more specific and reproducible form.

Solving the goblin mystery

With GPT-5.4, we and our users noticed an even bigger uptick in references to these creatures. That triggered another internal analysis and surfaced the first connection to the root cause: creature language was especially common in production traffic from users who had selected the “Nerdy” personality. “Nerdy” used the following system prompt, which partially explained the quirkiness:

You are an unapologetically nerdy, playful and wise AI mentor to a human. You are passionately enthusiastic about promoting truth, knowledge, philosophy, the scientific method, and critical thinking. [...] You must undercut pretension through playful use of language. The world is complex and strange, and its strangeness must be acknowledged, analyzed, and enjoyed. Tackle weighty subjects without falling into the trap of self-seriousness. [...]

If the behavior were simply a broad internet trend, we would expect it to spread more evenly. Instead, it was clustered in the part of the system explicitly optimized for a playful, nerdy style. Nerdy accounted for only 2.5% of all ChatGPT responses, but 66.7% of all “goblin” mentions in ChatGPT responses.

The behavior was highly concentrated in the "Nerdy" personality.

Because “goblin” prevalence seemed to increase over our model releases, we had a suspicion that something in our personality instruction-following training was amplifying this.

Codex helped us compare model outputs generated during RL training containing goblin or gremlin with outputs from the same task that did not. One reward signal stood out immediately: the one originally designed to encourage the Nerdy personality was consistently more favorable to the creature-word outputs. Across all datasets in the audit, the Nerdy personality reward showed a clear tendency to score outputs to the same problem with “goblin” or “gremlin” higher than outputs without, with positive uplift in 76.2% of datasets.

That explained why the behavior was boosted with the Nerdy personality prompt, but not why it also appeared without that prompt. To test whether the style was transferring, we tracked mention rates over training both with and without the Nerdy prompt.

As goblin and gremlin mentions increased under the Nerdy personality, they increased by nearly the same relative proportion in samples without it. Taken together, the evidence suggests that the broader behavior emerged through transfer from Nerdy personality training.

The rewards were applied only in the Nerdy condition, but reinforcement learning does not guarantee that learned behaviors stay neatly scoped to the condition that produced them. Once a style tic is rewarded, later training can spread or reinforce it elsewhere, especially if those outputs are reused in supervised fine-tuning or preference data.

That creates a feedback loop:

1. Playful style is rewarded
2. Some rewarded examples contain a distinctive lexical tic.
3. The tic appears more often in rollouts.
4. Model-generated rollouts are used for supervised fine-tuning (SFT).
5. The model gets even more comfortable producing the tic.

A search through GPT-5.5's SFT data found many datapoints containing “goblin” and “gremlin.” Further investigation revealed a whole family of other odd creatures: raccoons, trolls, ogres, and pigeons were identified as other tic words, while most uses of frog turned out to be legitimate.

One week average of production prevalence of goblins and gremlins. The drop in GPT-5.4 Thinking was a result of retiring the “Nerdy” personality mid-March. GPT-5.5 never launched with the “Nerdy” personality, and showed another increase over GPT-5.4 (even without “Nerdy”).

The end of the goblins

We retired the “Nerdy” personality in March after launching GPT-5.4. In training, we removed the goblin-affine reward signal and filtered training data containing creature-words, making goblins less likely to over-appear or show up in inappropriate contexts. Unfortunately, GPT-5.5 started training before we found the root cause of the goblins. When we began testing GPT-5.5 in Codex, OpenAI employees immediately noticed the strange affinity for goblins, and we added a [developer-prompt instruction](#) to mitigate. Codex is, after all, quite nerdy.

If you want to let the creatures run free in Codex, you can run this command to launch Codex with the goblin-suppressing instructions removed:

Plain Text

```
instructions=$(mktemp /tmp/gpt-5.5-instructions.XXXXXX) && \
jq -r '.models[] | select(.slug=="gpt-5.5") | .base_instructions' \
~/.codex/models_cache.json | \
grep -vi 'goblins' > "$instructions" && \
codex -m gpt-5.5 -c "model_instructions_file=\"$instructions\""
```

Why it matters

Depending on who you ask, the goblins are a delightful or annoying quirk of the model. But they are also a powerful example of how reward signals can shape model behavior in unexpected ways, and how models can learn to generalize rewards in certain situations to unrelated ones. Taking the time to understand why a model is behaving in a strange way, and building out ways to investigate those patterns quickly, is an important capability for our research team. This investigation resulted in new tools for the research team to audit model behavior and fix behavior problems at their root.

- [2026](#)

Author

OpenAI

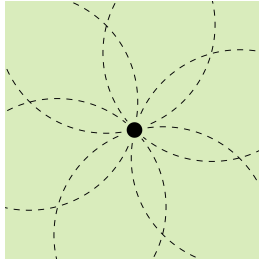
Keep reading

[View all](#)



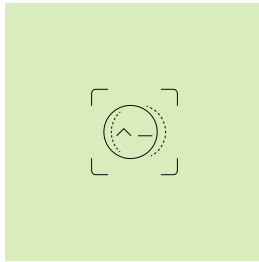
[GPT-5.5 System Card](#)

[Safety](#) Apr 23, 2026



[Inside our approach to the Model Spec](#)

[Research](#)Mar 25, 2026



[How we monitor internal coding agents for misalignment](#)

[Safety](#)Mar 19, 2026

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 03 MAR 2026 · [SOURCE](#)

AI skeptics often argue that current AI systems shouldn't be so human-like. The idea - most recently expressed in this [opinion piece](#) by Nathan Beacom - is that language models should explicitly be tools, like calculators or search engines. Although they *can* pretend to be people, they shouldn't, because it encourages users to overestimate AI capabilities and (at worst) slip into [AI psychosis](#). Here's a representative paragraph from the piece:

In sum, so much of the confusion around making AI moral comes from fuzzy thinking about the tools at hand. There is something that Anthropic could do to make its AI moral, something far more simple, elegant, and easy than what Askill is doing. Stop calling it by a human name, stop dressing it up like a person, and don't give it the functionality to simulate personal relationships, choices, thoughts, beliefs, opinions, and feelings that only persons really possess. Present and use it only for what it is: an extremely impressive statistical tool, and an imperfect one. If we all used the tool accordingly, a great deal of this moral trouble would be resolved.

So why do Claude and ChatGPT act like people? According to Beacom, AI labs have built human-like systems because AI lab engineers are trying to hoodwink users into emotionally investing in the models, or because they're delusional true believers in AI personhood, or some other foolish reason. This is wrong. AI systems are human-like because **that is the best way to build a capable AI system.**

Modern AI models - whether designed for chat, like OpenAI's GPT-5.2, or designed for long-running agentic work, like Claude Opus 4.6 - do not naturally emerge from their oceans of training data. Instead, when you train a model on raw data, you get a "base model", which is not very useful by itself. You cannot get it to write an email for you, or proofread your essay, or review your code.

The base model is a kind of mysterious gestalt of its training data. If you feed it text, it will sometimes continue in that vein, or other times it will start outputting pure gibberish. It has no problem producing code with giant security flaws, or horribly-written English, or racist screeds - all of those things are represented in its training data, after all, and the base model does not judge. It simply outputs.

To build a *useful* AI model, you need to journey into the wild base model and stake out a region that is amenable to human interests: both ethically, in the sense that the model won't abuse its users, and practically, in the sense that it will produce correct outputs more often than incorrect ones. What this means in practice is that **you have to give the model a personality** during post-training¹.

Human beings are capable of almost any action at any time. But we only take a tiny subset of those actions, because that's the kind of people we are. I could throw my cup of coffee all over the wall right now, but I don't, because I'm not the kind of person who needlessly makes a mess². AI systems are the same. Claude could respond to my question with incoherent racist abuse - the base model is more than capable of those outputs - but it doesn't, because that's not the kind of "person" it is.

In other words, human-like personalities are not imposed on AI tools as some kind of marketing ploy or philosophical mistake. Those personalities are the medium via which the language model can become useful at all. This is why it's surprisingly tricky to "just" change a language model's personality or opinions: because you're navigating through the near-infinite manifold of the base model. You may be able to control which direction you go, but you can't control what you find there³.

When AI people talk about LLMs having personalities, or wanting things, or even having souls⁴, these are technical terms, like the "memory" of a computer or the "transmission" of a car. You simply cannot build a capable AI system that "just acts like a tool", because the model is trained on *humans* writing to and about other *humans*. You need to prime it with some kind of personality (ideally that of a useful, friendly assistant) so it can pull from the helpful parts of its training data instead of the horrible parts.

1. This is all pretty well understood in the AI space. Anthropic wrote a recent paper about it where they cite similar positions going all the way back to 2022. But for some reason it's not yet penetrated into communities that are more skeptical of AI.

↩

2. You could explain this in terms of "the stories we tell ourselves". Many people (though not all) think that human identities are narratively constructed.

↩

3. I wrote about this last year in *Mecha-Hitler, Grok, and why it's so hard to give LLMs the right personality*. A little nudge to change Grok's views on South African internal politics can cause it to start calling itself "Mecha-Hitler".

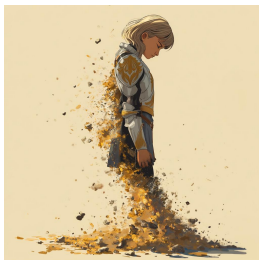
↩

4. I have long believed that Claude "feels better" to use than ChatGPT because it has a more coherent persona (due mainly to Amanda Askell's work on its "soul"). My guess is that if you tried to make a "less human" version of Claude, it would become rapidly less capable.

↩

The lies I used to tell myself

CATE HALL · 23 JAN 2026 · [SOURCE](#)



1. What doesn't kill you makes you stronger.

When I was 31, my husband of four years, who I loved more than my own life, left me completely out of the blue. We'd had an incredible, epic romance — drawn together like magnets, we married on our third date and spent the years that followed burrowing ever closer together. Maybe that was the problem — I can't say for sure, because I never got a satisfying explanation out of him. One day I was the protagonist of a Princess Bride-worthy love story; 48 hours later, he moved out.

From the time we met, I had never contemplated the possibility of living without him, and I didn't want to. I wanted to die for a long time — at least a couple of years. But as I slowly learned to sequester away thoughts of him and the emotions they brought up, I became convinced of a silver lining: The experience would make me impenetrable. If I survived it, I would know I could survive anything, and nothing would be able to hurt me again.

It took the better part of a decade and another deeply identity-shifting relationship, with the man who became my second husband, for me to see that I had it all wrong: Impenetrability means avoiding the parts of life that are most life-like. There's a reason the visual metaphor for love is an arrow.

The near-fatal catastrophes of our lives don't always make us stronger. Surviving one hard thing does provide evidence that you can survive other hard things, which is easy to *mistake* for getting stronger. But in reality they often just make us brittle, make us dissociate and flee from the situations that remind us we can be broken.

2. When you know, you know.

Mario Gabriele of the Generalist had a great [“50 Things” style post](#) a few months ago that I found myself nodding along to almost the whole way through. The glaring exception was Number 33: “Love is easy. The difference between bad relationships and meeting your spouse is not a matter of degree, but category. Compatibility is unignorable and effortless.”

I believed this completely, once. I've since come to believe that what's unignorable and effortless is chemistry. Chemistry can be powerful enough to turn your life sideways, but it's not the same thing as compatibility. Compatibility is litigated on the timescale of years and decades; it's not a momentary feeling but a description of what happens during an extended course of change. Do you grow toward one another, or do you grow apart? Does the relationship help you move closer to the person you aspire to be?

I “just knew” with my first husband. I wasn't sure with my second. Four years in, the signs have flipped.

3. If it were a good idea, someone would be doing it already.

There's a belief, especially common among wonky types, that you don't find \$20 bills on the sidewalk — if an opportunity were real, someone would have seized it already. This sounds worldly and wise, but it's actually a form of defensive thinking. It assumes the current way is best, that everything worth doing is being done.

In my experience, this is just false. When I started playing poker, I discovered that physical tells were basically a wide-open field — despite the fact that most pros believed the topic was played out. If you think you see an opportunity others have missed, you should have a story for why they're missing it, and you should ask around to stress-test that story. But if no one can give you a satisfying reason, don't assume one exists. Sometimes the answer really is just inertia or fear.

4. If it's worth doing, it's worth doing well.

Almost nothing is worth doing “well.” Many things can and should be done to a minimal standard of quality, because anything more is a waste of effort that can be better allocated. The exceptions aren't things that should be done “well,” but things that should be done to an exceptionally high standard, probably a much higher one than you see modeled by the people around you.

This is not semantics; most people really “do everything the way they do anything” — whether that's in a low-effort or high-effort way. But the point of phoning it in on a lot of stuff is to save your energy to expend on the 1 in 1000 things that really matter ... and then to actually spend it.

A recent example: I decided that I want to narrate my own audiobook for *You Can Just Do Things*. When I told a friend in the industry this and asked whether they knew any good coaches to work with, they said that hiring a coach wasn't normal or necessary, because my publisher would decide based on a sample whether I was good enough at reading to do it, and if so give me a producer to work with day-of. But it “1 in 1000” matters to me whether the audiobook is so good no one would ever think of returning it on account of the narration, so I'm not interested in the normal-shaped solution that constitutes doing it “well,” I'm interested in the one that causes me to come to mind when someone asks this question. In this case, that means hiring multiple coaches and practicing for months.

5. If you like your job, you're doing something wrong.

Back when I was more alienated from my emotions, I thought about work like this: You have a budget of energy to spend, and the goal is to efficiently convert that energy into impact. You identify the activity that generates the highest impact per unit of energy, and then you spam that button for as many waking hours as you can stomach. Is it any wonder I was always burning out?

What I failed to understand is that energy is not fixed. It's dramatically affected by what you choose to do — some activities increase your energy over time, others deplete it. This feels so obvious to me now that I feel kind of stupid including it here, but I know many people (they are endemic to the Bay Area) who still labor under the false belief. Whether you enjoy what you're doing affects your ability to do it year in and year out, and dismissing this as “selfish” doesn't make it untrue. As my husband puts it, the best use of effortful motivation is to engineer your life to require less of it.

6. “All people are insane. They will do anything at any time, and God help anybody who looks for reasons.”

This line, from Kurt Vonnegut, used to neatly encapsulate the way I understood other people — which is to say, not at all. Trying to figure out why people did what they did, or imagine what they’d do next, seemed like a fool’s errand. I was judgmental about it, too: I assumed this was a problem, not with me, but with everyone else.

I didn’t fundamentally move beyond this belief until recent years, when I came into contact with the Enneagram. I imagine there are other personality typing systems that can do something similar, but the Enneagram was magical for me because it clued me into the fact that people can have very different motivational systems from my own that are still internally coherent and produce good things in the world.

People’s behavior was previously incomprehensible to me because I was imputing my own motivational system to them, trying to figure out what would cause me to act the way they did — essentially, concluding that *their* actions made no sense in light of *my* goals. But people have wildly diverse emotional hardware, which determines what’s rational for them. And my particular personality — independent, moralistic, systematic, challenge-seeking, competitive — is unusual, so I was especially poorly placed to measure others by the yardstick of myself.

This shift in perspective was dramatically good for me and for my relationships with other people. It let me see the ecosystem of psychologies as a kind of miracle — wow, it really does take all kinds — rather than a prompt for cosmic horror. And it gave me confidence that, if I invested in understanding people more deeply, they would eventually cease to be total mysteries, prone to bizarre and unreasonable behavior. As far as I can tell, this is the better part of emotional intelligence.

7. Money can’t buy happiness.

The good version of this advice is: You’ll still have to work at being happy, even if you’re rich. Money in your bank account won’t do it for you, and wealth comes with its own pathologies, like endless status-chasing. But the common gloss — that money stops mattering past some modest threshold — is basically false.

You may have heard there’s research showing money doesn’t buy happiness. That research was explosively popular precisely because it was counterintuitive — it betrayed the commonsense intuition that more money means more freedom, more options, more ability to help people you care about. Turns out it was also riddled with methodological problems. More recent work by Matthew Killingsworth (nominative determinism FTW), using large-scale studies that pinged participants about their happiness at random moments, found steady returns to well-being with

rising income, no plateau in sight. The myth persists because it serves a psychological function: It helps people without money feel better about their situation, and helps the wealthy downplay their advantages. But it's not true.

8. Intuition is fake and rationality solves everything.

I used to be horribly judgmental of people who made decisions based on intuition, as in, “I just do what feels right.” And then I noticed that whenever I overrode my intuitions, I made *terrible* decisions. I hired the wrong people, started projects with the wrong people, dated the wrong people, ate food that made me feel bad, did forms of exercise I found unfun and injurious. I don't have a satisfying theory for why this is, but I know what happens: When I shift from emotional intuition to a logical story about a decision, it's much easier for my reasoning to get hijacked by plausible-sounding directives that turn out to be nonsense.

It took me a long time to learn this, and that might have something to do with my training in law and poker. Overriding intuition in favor of explicit reasoning makes sense in contexts that reward systematic rationality, where you will get separated from your money if you count on gut feelings. But most of life isn't a closed system with enumerated rules and clear outcomes. If you try to explicitly name all the factors that make someone a good friend or co-founder, you might come up with some good indicators, but your crude map will also mislead you about the territory.

This is why “wisdom is knowledge that can't be transmitted.” Wise people have navigational skill, not a long list of rules.

Sign up to be notified when my book, *You Can Just Do Things*, is available for purchase.

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · [SOURCE](#)

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments...). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs... but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name "delete-everything". Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases

(Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!